

/*

----- French -----

Exercice 2 :

Un carré (matrice) est dit magique, si la somme de ses lignes, ses colonnes et ses diagonales est la même.

- 1) Écrire une fonction `int** saisie(int** M, int dimension)` qui réserve la place exacte pour la matrice M de taille `dimension`, lit ses éléments au clavier.
- 2) Ecrire une fonction `Affiche(int ** M, int dimension)` `dimension)`, qui affiche, sous forme matricielle, les éléments de M.
- 3) Ecrire une fonction `carre_magique(int **M, int dimension)` qui vérifie si le carré est magique.
- 4) Ecrire un programme principal de test.

----- English -----

Exercise 2:

A square (matrix) is said to be magic if the sum of its rows, columns and diagonals is the same.

- 1) Write a function `int** input(int** M, int dimension)` which reserves the exact place for the matrix M of dimension size, reads its elements from the keyboard.
- 2) Write a function `Display(int ** M, int dimension)`, which displays, in matrix form, the elements of M.
- 3) Write a function `square_magic(int **M, int dimension)` which checks if the square is magic.
- 4) Write a main test program.

*/

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

/* Function in use */

int get_rows_cols(int rows, int cols);
int **allocate(int **matrix, int length);
void get_matrix_values(int **matrix, int length);
void display(int **matrix, int length);
bool square_magic(int **matrix, int length);

int main()
{
    int **matrix, rows, cols, length;
    bool magic;
    length = get_rows_cols(rows, cols);
    matrix = allocate(matrix, length);
    get_matrix_values(matrix, length);
    display(matrix, length);
    magic = square_magic(matrix, length);

    if (magic)
        printf("\nvvv The given Matrix is a MAGIIC SQUARE\n");
    else
        printf("\n!!! The given Matrix is NOT a MAGIIC SQUARE\n");

    printf("\n");
    return 0;
}

int get_rows_cols(int rows, int cols)
{
    int length, choice;
    printf("Rows = ");
    scanf("%d", &rows);
    printf("Columns = ");
    scanf("%d", &cols);
    if (rows != cols)
    {
        printf("Rows must be equal to the Columns\n");
    }
}

```

```

        printf("While Rows not equal to the Columns, SQUARE MAGIC is never going to
happen.\n");
        printf("Retry click 1\t\t");
        printf("Quit click 2");
        printf("\n-----Choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            main();
            break;

        case 2:
            printf("Exiting...\n");
            exit(0);
            break;
        }
    }
else
{
    length = cols;
    return length;
}
}

```

```

int **allocate(int **matrix, int length)
{
    matrix = (int **)malloc(length * sizeof(int *));
    for (int i = 0; i < length; i++)
        matrix[i] = (int *)malloc(length * sizeof(int));
    return matrix;
}

```

```

void get_matrix_values(int **matrix, int length)
{
    printf("\n_____ \nEnter values of the matrix\n-----
-----\n ");
    for (int i = 0; i < length; i++)

```

```
    for (int j = 0; j < length; j++)
        scanf("%d", (*(matrix + i) + j));
```

```
}
```

```
void display(int **matrix, int length)
```

```
{
```

```
    printf("\n\t***** The given matrix *****\n");
```

```
    for (int i = 0; i < length; i++)
```

```
    {
```

```
        for (int j = 0; j < length; j++)
```

```
            printf("%d\t", (*(matrix + i) + j));
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
bool square_magic(int **matrix, int length)
```

```
{
```

```
    int i, j, sum = 0, nextSum = 0;
```

```
    bool magic = true;
```

```
    printf("\n_____ For Rows _____\n");
```

```
    for (j = 0; j < length; j++)
```

```
    {
```

```
        sum += matrix[0][j];
```

```
        printf("\n/+/+/+/+/+\ti)0 = j)%d | result = %d", j, sum);
```

```
    }
```

```
    printf("\n");
```

```
    i = 1;
```

```
    while (magic && i < length)
```

```
    {
```

```
        nextSum = 0;
```

```
        for (j = 0; j < length; j++)
```

```
        {
```

```
            nextSum += matrix[i][j];
```

```
            printf("\n/+/+/+/+/+\ti)%d = j)%d | result = %d", i, j, nextSum);
```

```
        }
```

```
        printf("\n");
```

```
        if (sum != nextSum)
```

```
            magic = false;
```

```

        i++;
    }

    printf("\n_____ For Cols _____\n");
    j = 0;
    while (magic && j < length)
    {
        nextSum = 0;
        for (i = 0; i < length; i++)
        {
            nextSum += matrix[i][j];
            printf("\n/+/+/+/+/\ti)%d = j)%d | result = %d", i, j, nextSum);
        }
        printf("\n");
        if (sum != nextSum)
            magic = false;
        j++;
    }
    if (magic)
    {
        nextSum = 0;
        for (i = 0; i < length; i++)
            nextSum += matrix[i][i];
        if (sum != nextSum)
            magic = false;
        nextSum = 0;
        for (i = 0; i < length; i++)
            nextSum += matrix[i][length - 1 - i];
        if (sum != nextSum)
            magic = false;
    }

    printf("\n_____ Final Decision\n\n");

    if (sum == nextSum)
        printf("\nSum of each row or column %d.", sum);
    return magic;
}

```

/*

Edit by: MOHAMMED ELIDRISSI LAOUKILI
#idriCoding | #idriLogic | #idri
2021/2022

لا اله الا الله

*/